Great question — this is where a lot of course projects go wrong by testing on the same data they trained on. Here's exactly how to think about it:

The Core Rule

Never test on the same location you trained on. With geospatial data, a random 80/20 split of patches is not enough — nearby patches share the same landscape texture, so the model memorises the area, not the pattern. You need a **spatial split**.

Your Train / Validation / Test Split

Split	Location	Purpose
Train (60%)	Peenya Industrial Area	Model learns green belt vs. built-up patterns
Validation (20%)	Bommasandra Industrial Area	Tune hyperparameters, pick best checkpoint
Test (20%)	Nanjangud Industrial Area	Final evaluation — touch this only once

This works perfectly for you because you already have 3 AOIs. Peenya is the biggest so it gives the most training patches, and Nanjangud is geographically different enough (Mysuru region vs Bengaluru) to be a genuine unseen test.

For Testing Specifically — Use These

Option 1 — Your own held-out data (minimum viable)

The Nanjangud Sentinel-2 imagery you export from GEE, clipped to KGIS boundaries, with manually annotated masks in QGIS. This is what you must have at minimum for a course project.

Option 2 — Public benchmark datasets (adds credibility)

These are ready-made datasets with labels — no annotation needed:

Dataset	Task	Why use it
LEVIR-CD	Change detection	637 bitemporal image pairs, building change labels — perfect for testing your Siamese U-Net

Dataset	Task	Why use it
WHU-CD	Change detection	High-res aerial imagery, building footprint changes — good cross-domain test
BigEarthNet-S2	Multi-label land cover	590k Sentinel-2 patches with CORINE labels — use a vegetation subset for segmentation testing
DSTL Satellite Imagery (Kaggle)	Multi-class segmentation	Includes vegetation, water, buildings — directly comparable to your task
GID (Gaofen Image Dataset)	Land use segmentation	5 classes including vegetation and built-up — large scale, high quality

Option 3 — Temporal robustness test (most impressive for evaluation)

Run your trained model on **the same factories across different seasons** — monsoon (Jun–Sep) vs. dry (Nov–Feb) — without retraining. If F1 stays stable, your model generalises. If it drops badly in monsoon season due to cloud/NDVI variation, you know where to improve. This is a test evaluators rarely see in course projects and it directly demonstrates real-world robustness.

Practical Recommendation

For your course project, do this in order:

1. **Primary test** — Nanjangud held-out data with your own annotations. Non-negotiable.
2. **Benchmark test** — Download LEVIR-CD and run your change detection model on it. Zero annotation effort, and citing a public benchmark immediately makes your evaluation look professional.
3. **Seasonal test** — Export one extra Sentinel-2 image from Peenya during monsoon season and run inference. Report whether F1 drops and by how much.